

Virtual Machine in C

Mykyta ‘Nick’ Khomiakov

May 10, 2026

Contents

1	Abstract	3
2	Project Setup	5
2.1	Prerequisites	5
2.2	File Directory	6
2.3	Build	7
2.4	Running the VM	8
2.5	Separation of Concerns	9
3	Instruction Set Architecture	10
3.1	Stack Manipulation	10
3.2	Arithmetic	11
3.3	Logic and Comparison	11
3.4	Control Flow	12
3.5	Registers	13
3.6	I/O and Control	14
3.7	Bytecode Encoding Summary	15
4	Example Programs	15
4.1	Arithmetic Expressions	15
4.2	Conditional Execution	16
4.3	Loops	17
4.4	Functions	18
4.5	Register-Based Patterns	19
4.6	Patterns and Idioms Summary	20
5	Implementation Notes	21
5.1	The <code>push</code> and <code>pop</code> Primitives	21
5.2	A Latent Bug in <code>op_div</code>	22
5.3	A Latent Bug in <code>op_store</code>	23
5.4	Flat Integer Bytecode	23
5.5	Zero-Initialisation of the VM Struct	24
5.6	Separation of the Operand and Call Stacks	24

5.7	No Bounds Checking on Jump Targets	24
5.8	The REPL's Auto-Reset Behaviour	25
6	Comparison with Real-World Virtual Machines	25
6.1	The Java Virtual Machine	25
6.2	CPython	26
6.3	WebAssembly	26
6.4	Summary of Differences	27
7	Conclusions	27

1 Abstract

This project presents the design and implementation of a stack-based virtual machine (VM) written entirely in C, conforming to the C11 standard [1]. The machine supports a custom instruction set of 25 opcodes spanning five categories: arithmetic, logic and comparison, control flow, stack manipulation, and register operations. Alongside the VM core, the project includes a purpose-built lexer, a single-pass assembler, and an interactive Read-Evaluate-Print Loop (REPL) shell — all implemented from scratch with no external dependencies beyond the C standard library.

Motivation

The primary motivation for this project was a curiosity-driven investigation into how virtual machines, interpreters, and CPUs work at a fundamental level. Rather than treating a programming language runtime as a black box, the goal was to build one from the ground up — making explicit every design decision that is ordinarily hidden: how instructions are encoded, how operands are passed implicitly through a stack, how control flow is redirected, and how a call stack enables function abstraction.

Architecture Overview

The VM follows a classical *fetch–decode–execute* cycle [2]. On each tick, the instruction pointer (`ip`) reads the next integer from a flat bytecode array (the *fetch* phase). The integer is dispatched through a `switch` statement (the *decode* phase), which routes execution to the appropriate handler function (the *execute* phase). The loop continues until a `HALT` instruction sets the running flag to zero.

The complete runtime state is encapsulated in a single `VM` struct:

- **Operand stack** (`int stack[256]`): The primary working memory. All arithmetic, logic, and comparison operations read their inputs from and write their results to this stack. The stack pointer `sp` always points to the next free slot.
- **General-purpose registers** (`int registers[16]`): Sixteen integer registers (`R0–R15`), accessible via the `LD` and `ST` instructions. Registers provide a simple means of persisting values across stack operations without re-pushing them every time.
- **Call stack** (`int call_stack[128]`): A dedicated return-address stack, separate from the operand stack, supporting up to 128 levels of nested function calls. The `CALL` instruction saves the current `ip` here before jumping; `RET` restores it.
- **Instruction pointer** (`ip`) and **call stack pointer** (`csp`): Integer indices into the bytecode array and call stack, respectively.

No global variables or hidden singletons exist anywhere in the codebase. Every function receives a pointer to the VM struct, making the state explicit and the code straightforward to reason about.

Why Stack-Based?

Most modern physical CPUs are register-based: they shuttle values between a fixed set of named registers (`eax`, `rbx`, `r0`, etc.) before operating on them. A stack-based VM instead uses an implicit operand convention — instructions always read from and write to the top of the stack, so no operand addresses need to be encoded in the instruction stream. This choice was made deliberately for the following reasons:

- **Implementation simplicity.** There is no register-allocation problem to solve. Every opcode knows exactly where its inputs are (the top of the stack) and where to leave its result (back on the top). The interpreter loop reduces to a clean `switch` over a flat integer array with no complex decode logic.
- **Compact bytecode.** Because operands are implicit, most instructions occupy a single integer word. Only instructions that require a literal value — `PUSH`, `JMP`, `JZ`, `JNZ`, `CALL`, `LD`, and `ST` — consume an additional word for that operand. This keeps the bytecode representation minimal and the assembler trivial to implement.
- **Portability.** The VM state is composed entirely of plain `int` arrays and integers. There are no platform-specific intrinsics, no inline assembly, and no OS-level system calls beyond `printf`. The same source compiles and behaves identically on Linux, macOS, and Windows via CMake.
- **Pedagogical clarity.** Stack machines have a direct correspondence with how arithmetic expressions are evaluated in postfix (Reverse Polish Notation) form [3]. The execution model is easy to trace step by step, making this architecture a natural fit for studying how high-level expressions are lowered into a sequence of low-level operations — the same transformation performed by production compilers targeting the JVM [4] or the .NET CLR [5].

The primary trade-off is instruction count: a stack machine often requires additional instructions (`DUP`, `SWAP`) to reorder operands that a register machine could address directly. This mirrors the philosophy of Reduced Instruction Set Computing (RISC) [6], where a smaller, more uniform instruction set simplifies the processor at the cost of requiring more instructions to perform equivalent work. For a teaching and experimentation platform, this cost is entirely acceptable.

Toolchain: Lexer, Assembler, and REPL

Source text typed at the interactive prompt passes through three sequential stages before execution:

1. **Lexer** (`lexer.c`): Receives a raw line of text. Strips inline comments (any text following a `;` character), uppercases the first token to form the mnemonic, and parses up to two integer operands in either decimal or hexadecimal (`0x...`) notation. Produces a single **Token** struct.
2. **Assembler** (`assembler.c`): Looks up the mnemonic in a static table of 25 entries, each recording the instruction's opcode integer and expected operand count. Emits the opcode word followed by any operand words into a flat **Program** buffer (up to 1024 words). Issues a warning if the operand count does not match the table entry.
3. **VM** (`vm.c`): Receives the populated **Program** buffer, resets a fresh **VM** struct, and enters the fetch–decode–execute loop until **HALT** or a runtime error.

The REPL itself provides four shell commands — **RUN**, **LIST**, **RESET**, and **HELP** — that are intercepted before assembly and never enter the bytecode stream.

Error Handling

The VM detects and reports six categories of runtime error, printing a descriptive message and halting cleanly in each case: stack overflow (operand stack exceeds 256 slots), stack underflow (a pop or binary operation finds fewer values than required), division by zero (**DIV** with a zero divisor), invalid register index (**LD** or **ST** with an index outside 0–15), call stack overflow (**CALL** nested beyond 128 frames), and call stack underflow (**RET** with no matching **CALL**). In every case, the VM sets `running = 0` and returns normally — no crashes, no undefined behaviour.

Separation of Concerns

Each functional category of opcode lives in its own translation unit under `src/operations/`: `maths.c`, `logic.c`, `flow.c`, `registers.c`, and `stack.c`. The central dispatcher in `vm.c` contains only `case` labels; it delegates immediately to the relevant handler. Adding a new opcode requires touching at most three files: the `enum` in `vm.h`, the implementation file, and the mnemonic table in `assembler.c`. No other file needs to change. This modularity was a deliberate architectural goal, ensuring the codebase remains comprehensible as the instruction set grows.

2 Project Setup

2.1 Prerequisites

The project is built with **CMake 4.1** [7] and targets the **C11** language standard [1]. Before building, the following tools must be present on the host machine:

- **C compiler with C11 support** — either GCC (`gcc`) or Clang (`clang`) will work. The compiler must accept the `-std=c11` flag. GCC 5+ and Clang 3.3+ both satisfy this requirement.
- **CMake 4.1 or later** — used to generate the native build system files. CMake is platform-independent and produces Makefiles on Linux, Xcode project files on macOS, and Visual Studio solution files on Windows. Download from cmake.org or install via a package manager.
- **GNU Make** (Linux/macOS only) — required to consume the Makefile that CMake generates on Unix-like systems. On macOS, Make ships with the Xcode Command Line Tools (`xcode-select --install`). On Windows, CMake targets MSBuild or Ninja instead, so Make is not needed.

Recommended development environments by platform:

- **Windows** — Visual Studio 2022 (Community edition is free). CMake integrates natively and can target the MSVC toolchain or the bundled Clang front-end.
- **macOS / Linux** — JetBrains CLion, which has built-in CMake support and a bundled GDB/LLDB debugger. VS Code with the *CMake Tools* and *C/C++* extensions is a lightweight alternative.

2.2 File Directory

The repository is organised into three top-level directories: `src/` for all translation units, `include/` for their corresponding headers, and `docs/` for the L^AT_EX paper. Source files are further grouped by responsibility:

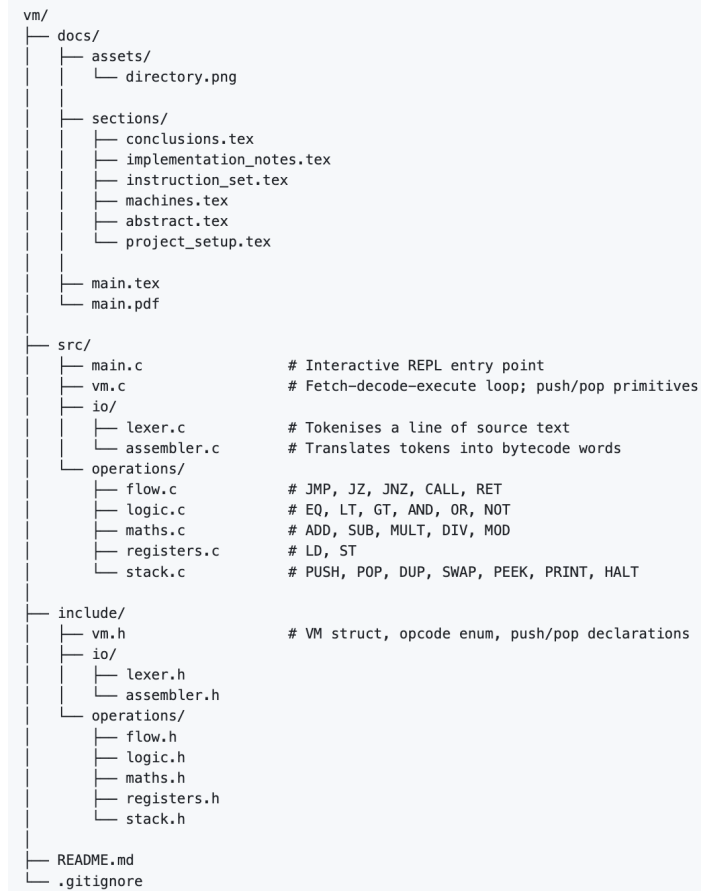


Figure 1: File Directory of the project

Every `.c` file under `src/operations/` is a self-contained translation unit. It includes only its own header and `vm.h`; it has no knowledge of sibling modules. This layout enforces at the filesystem level the same separation of concerns described in the architecture section.

2.3 Build

Option A — CMake (recommended)

CMake is the canonical way to build the project. From the repository root:

```
# 1. Create and enter a build directory
mkdir build && cd build
```

```
# 2. Configure (generates Makefile / solution)
```

```
cmake ..
```

```
# 3. Compile
cmake --build .
```

The resulting executable is placed at `build/virtual_machine` on Unix-like systems and `build\Debug\virtual_machine.exe` on Windows.

Option B — Direct GCC / Clang invocation

For environments without CMake, the project can be compiled with a single command from the repository root:

```
gcc -std=c11 -Wall -Wextra -Iinclude \
  src/main.c src/vm.c \
  src/io/lexer.c src/io/assembler.c \
  src/operations/math.c src/operations/stack.c \
  src/operations/flow.c src/operations/logic.c \
  src/operations/registers.c \
  -o vm
```

Replace `gcc` with `clang` if that is the installed compiler. The flags `-Wall -Wextra` enable a broad set of compiler warnings; the project compiles cleanly at both levels.

2.4 Running the VM

Once compiled, launch the interactive assembler from the directory containing the binary:

```
./vm          # Unix-like systems
vm.exe        # Windows
```

The REPL greets the user and waits for input. A brief example session — pushing two values, adding them, printing the result, and halting — looks like this:

```
Virtual Machine -- interactive assembler
Type HELP for usage, QUIT to exit.
```

```
> PUSH 10
  [@ word 0]
> PUSH 32
  [@ word 2]
> ADD
  [@ word 4]
> PRINT
  [@ word 5]
> HALT
  [@ word 6]
>
```



```

--- Running (7 word(s)) ---
42
--- Done ---

```

The shell also accepts four meta-commands that are processed by the REPL and never assembled into the program:

Command	Effect
RUN	Assemble and execute the current buffer
LIST	Print each word with its bytecode index
RESET	Clear the program buffer
HELP	Display the full opcode reference
QUIT / EXIT	Exit the program

A blank line is equivalent to RUN. Lines beginning with ; are treated as comments and stripped before lexing.

2.5 Separation of Concerns

The codebase is structured around a strict *separation of concerns*: no module knows about the internals of another module it does not directly depend on. The dependency graph is deliberately shallow:

- **Operation modules** (`maths`, `logic`, `flow`, `registers`, `stack`) depend only on `vm.h`. They are entirely unaware of one another.
- `vm.c` depends on all five operation headers, but only to dispatch via `case` labels. It contains no arithmetic or logic of its own.
- `assembler.c` depends on `vm.h` (for the opcode enum) and `lexer.h` (for the `Token` type). It knows nothing about runtime execution.
- `lexer.c` depends on nothing beyond the C standard library. It is the most isolated module in the project.
- `main.c` is the sole integration point, pulling together the lexer, assembler, and VM.

A practical consequence of this design: adding a new opcode requires changes to at most three files — the `enum` in `vm.h`, the relevant operation source file, and the mnemonic table in `assembler.c`. No other file needs to be touched, and the REPL automatically recognises the new instruction without any further modification.

3 Instruction Set Architecture

This section documents every opcode supported by the VM. Instructions are grouped into five categories matching the source layout: stack manipulation, arithmetic, logic and comparison, control flow, and registers. Each entry records the mnemonic, its stack effect using standard notation, a description of the operation, and any runtime error conditions it can trigger.

Notation

Stack effects are written in the form **before** `--` **after**, where the rightmost element is the top of the stack. For example, `a b -- (a+b)` means two values are consumed and one is produced. The symbol `--` separates the state before the instruction executes from the state after. An empty left-hand side means the instruction pushes without consuming; an empty right-hand side means it consumes without pushing [8, 9].

Opcodes that require a literal argument encoded in the bytecode stream carry it as an immediate word following the opcode. These are marked with `<n>` (an integer literal) or `<addr>` (a bytecode word index) in the mnemonic column.

3.1 Stack Manipulation

These instructions operate directly on the shape of the operand stack. They move, duplicate, or discard values without performing any computation on them.

Mnemonic	Stack Effect	Description
PUSH <code><n></code>	<code>-- n</code>	Push the integer literal <code>n</code> onto the stack.
POP	<code>a --</code>	Discard the top value.
DUP	<code>a -- a a</code>	Duplicate the top value; both copies remain.
SWAP	<code>a b -- b a</code>	Swap the top two values.
PEEK	<code>a -- a</code>	Print the top value without consuming it.

PUSH is the only way to introduce a literal integer into the program. Its operand is encoded as the word immediately following the opcode in the bytecode stream, so `PUSH 42` assembles to the two-word sequence `[PUSH, 42]`.

DUP is used when a value must both be tested (consumed by a comparison or jump) and retained for the next iteration of a loop. A typical countdown idiom reads:

```
DUP          ; keep a copy for the jump test
JZ  <end>    ; consume the copy -- exit if zero
...
JMP <top>    ; loop back to DUP
```

SWAP is necessary when the order of operands matters for a non-commutative operation. Because `b` is popped before `a`, the code sequence `PUSH 10 / PUSH`

3 / SUB computes $10 - 3$, not $3 - 10$. SWAP lets a program correct the order if operands were pushed in the wrong sequence.

PEEK is a diagnostic aid. It prints the top of the stack to standard output but leaves the stack unchanged, making it useful for observing intermediate values during development without disturbing the program's data.

Error conditions: POP, DUP, and PEEK halt with *stack underflow* if the stack is empty. SWAP halts with *stack underflow* if fewer than two values are present. PUSH halts with *stack overflow* if the stack already holds 256 values.

3.2 Arithmetic

Arithmetic instructions each consume two values from the top of the stack and push a single result. The value pushed second is treated as the left-hand operand and the value pushed first (i.e., the top) as the right-hand operand, consistent with the natural push order.

Mnemonic	Stack Effect	Description
ADD	a b -- (a+b)	Integer addition.
SUB	a b -- (a-b)	Integer subtraction.
MULT	a b -- (a*b)	Integer multiplication.
DIV	a b -- (a/b)	Integer division (truncates toward zero).
MOD	a b -- (a%b)	Remainder after integer division.

All five instructions operate on signed 32-bit integers (the C `int` type on all target platforms). Overflow wraps silently under standard C integer arithmetic rules; the VM does not detect it.

DIV performs truncating integer division: $7 \div 2 = 3$, and $-7 \div 2 = -3$. The result is always rounded toward zero, matching C's built-in / operator for signed integers.

MOD computes the remainder consistent with DIV: the sign of the result matches the sign of the dividend, so $-7 \% 2 = -1$.

Error conditions: All five instructions halt with *stack underflow* if fewer than two values are present. DIV additionally halts with *division by zero* if the divisor (top of stack) is zero before the pop.

3.3 Logic and Comparison

Comparison instructions treat any non-zero integer as *true* and zero as *false*. They consume their operands and push either 1 (true) or 0 (false), allowing their results to be used directly as conditions for the conditional jump instructions JZ and JNZ.

Mnemonic	Stack Effect	Description
EQ	a b -- (a==b)	Push 1 if a equals b, else 0.
LT	a b -- (a<b)	Push 1 if a is less than b, else 0.
GT	a b -- (a>b)	Push 1 if a is greater than b, else 0.
AND	a b -- (a&& b)	Push 1 if both a and b are non-zero, else 0.
OR	a b -- (a b)	Push 1 if either a or b is non-zero, else 0.
NOT	a -- (!a)	Push 1 if a is zero; push 0 otherwise.

EQ, LT, and GT provide the full set of binary comparisons needed to express any relational condition. Less-than-or-equal ($a \leq b$) can be expressed as GT NOT (negate the greater-than result); greater-than-or-equal as LT NOT; not-equal as EQ NOT.

AND and OR implement *logical* (not bitwise) conjunction and disjunction. Any non-zero value is treated as true, so AND of 5 and 3 yields 1, not 1 (bitwise). This matches the semantics of C's && and || operators.

A typical guarded conditional pattern — execute a block only if two conditions both hold — looks like:

```
; stack: ... a b
AND          ; stack: ... (a && b)
JZ <skip>    ; jump past the block if false
... ; body
<skip>:
```

Error conditions: All binary instructions (EQ, LT, GT, AND, OR) halt with *stack underflow* if fewer than two values are present. NOT halts with *stack underflow* if the stack is empty.

3.4 Control Flow

Control flow instructions alter the instruction pointer (ip), enabling loops, conditionals, and function calls. All jump targets are expressed as absolute word indices into the bytecode array — the same indices reported by the LIST shell command.

Mnemonic	Stack Effect	Description
JMP <addr>	--	Unconditional jump to word index addr .
JZ <addr>	a --	Pop a; jump to addr if a is zero.
JNZ <addr>	a --	Pop a; jump to addr if a is non-zero.
CALL <addr>	--	Push return address to call stack; jump to addr .
RET	--	Pop return address from call stack; resume there.

JMP sets ip to the given word index unconditionally. Because the assembler emits each instruction's opcode at a known word offset (reported by [@ word N] in the REPL), jump targets can be calculated by inspection. Note that JMP itself

occupies two words (opcode + operand), so a backward jump to the instruction at word 4 requires the operand 4, not an offset.

JZ and JNZ are the conditional branches. They always consume the top of the stack regardless of whether the jump is taken. This means the condition value must be produced fresh on each iteration of a loop — typically via DUP before a comparison — if the underlying counter or flag must be preserved.

CALL saves the address of the instruction *immediately following* the CALL opcode word and its operand (i.e., ip after both words have been consumed) onto the call stack, then sets ip to the target address. RET restores ip from the top of the call stack, resuming execution at the saved address. This mechanism supports up to 128 levels of nested calls before the call stack overflows.

Parameters are conventionally passed on the operand stack, pushed by the caller before CALL and consumed by the function body. Return values are left on the operand stack by the function before RET. This convention is not enforced by the hardware; it is a programming discipline.

An example of a simple function call that doubles its argument:

```
; --- caller ---
PUSH 7      ; word 0: push argument
CALL 6      ; word 2: call function at word 6; return
             address = word 4
PRINT      ; word 4: print the return value
HALT      ; word 5

; --- function body (starts at word 6) ---
DUP         ; word 6: duplicate argument (a a)
ADD         ; word 7: a + a = 2a
RET         ; word 8: return to word 4
```

Output: 14

Error conditions: JZ and JNZ halt with *stack underflow* if the stack is empty. CALL halts with *call stack overflow* if 128 frames are already active. RET halts with *call stack underflow* if there is no matching CALL on the call stack. No bounds checking is performed on jump targets; jumping to an out-of-range address produces undefined behaviour.

3.5 Registers

The VM provides sixteen general-purpose integer registers, addressed by index 0 through 15. Registers hold their values across instructions and are initialised to zero at VM startup. They offer a way to store and retrieve values without keeping them permanently on the operand stack.

Mnemonic	Stack Effect	Description
ST <reg>	a --	Pop a and store it in register reg.
LD <reg>	-- a	Push the current value of register reg.

ST is a destructive write: the value is removed from the operand stack and placed in the register. LD is non-destructive with respect to the register: the register retains its value after the push. A register can be read any number of times with repeated LD instructions.

Registers are particularly useful for loop counters and function arguments that must survive multiple stack operations. Rather than duplicating a value repeatedly across loop iterations, it can be stored once in a register and reloaded as needed:

```
PUSH 10
ST 0      ; register 0 = 10 (loop limit)
PUSH 1
ST 1      ; register 1 = 1 (counter, starts at 1)

; loop body (word 8)
LD 1      ; push counter
PRINT     ; print it
LD 1      ; push counter again
PUSH 1
ADD       ; counter + 1
ST 1      ; store updated counter
LD 1      ; push counter for comparison
LD 0      ; push limit
GT        ; counter > limit?
JNZ 8     ; if not, repeat
HALT
```

Output: 1, 2, ..., 10

Error conditions: Both ST and LD halt with *invalid register* if the register index is outside the range 0–15. ST additionally halts with *stack underflow* if the stack is empty when the store is attempted.

3.6 I/O and Control

Mnemonic	Stack Effect	Description
PRINT	a --	Pop the top value and print it to standard output.
HALT	--	Stop execution; set the running flag to zero.

PRINT is the VM's sole output instruction. It pops the top value, formats it as a signed decimal integer, and writes it to `stdout` followed by a newline. It is intentionally destructive: if the value must be retained, DUP should precede it.

HALT terminates the fetch–decode–execute loop cleanly. Every well-formed program must end with HALT; without it, the VM will continue reading beyond the end of the bytecode array, causing undefined behaviour.

3.7 Bytecode Encoding Summary

The table below summarises the word width of each instruction as it appears in the flat bytecode array. Instructions with an operand occupy two words; all others occupy one.

Instruction(s)	Words	Operand
ADD, SUB, MULT, DIV, MOD	1	—
EQ, LT, GT, AND, OR, NOT	1	—
POP, DUP, SWAP, PEEK	1	—
PRINT, HALT, RET	1	—
PUSH <n>	2	Integer literal
JMP <addr>, JZ <addr>, JNZ <addr>	2	Word index
CALL <addr>	2	Word index
LD <reg>, ST <reg>	2	Register index (0–15)

Because all values are plain C ints, opcodes and operands share the same representation. The distinction between an opcode word and a data word is purely positional: the VM knows from context (specifically, from the `operand_count` field in the assembler’s mnemonic table) how many words following an opcode are operands rather than the next opcode.

4 Example Programs

This section presents a series of complete programs written for the VM, arranged in order of increasing complexity. Each example is self-contained: it can be typed into the interactive REPL exactly as shown and will produce the stated output. The intent is to demonstrate not just individual opcodes in isolation, but the idiomatic patterns that emerge when combining them — loops, conditionals, register-based storage, and function calls.

4.1 Arithmetic Expressions

The simplest programs evaluate arithmetic expressions and print their result. Because the VM uses a stack, all expressions are written in postfix (Reverse Polish) order: operands are pushed first, then the operator is applied.

Computing $(3 + 4) \times 5$

```
PUSH 3
PUSH 4
ADD      ; stack: 7
PUSH 5
MULT     ; stack: 35
PRINT   ; prints 35
HALT
```

Output: 35

Integer division with remainder

The following program computes both the quotient and the remainder of $17 \div 5$ in a single pass, using DUP to avoid pushing the operands twice.

```
PUSH 17
PUSH 5
DUP          ; stack: 17 5 5
ST 0         ; save divisor in R0 - stack: 17 5
DIV          ; stack: 3    (quotient)
PRINT        ; prints 3
PUSH 17
LD 0         ; reload divisor - stack: 17 5
MOD          ; stack: 2    (remainder)
PRINT        ; prints 2
HALT
```

Output: 3, 2

4.2 Conditional Execution

Conditional logic is expressed by combining a comparison opcode (EQ, LT, GT) with a conditional branch (JZ or JNZ). The comparison leaves a boolean (0 or 1) on the stack, which the branch then consumes.

Absolute value

The program reads a value (hard-coded here as -7) and prints its absolute value.

```
PUSH -7
DUP          ; keep a copy for the sign check - stack: -7 -7
PUSH 0
LT           ; is -7 < 0? pushes 1 - stack: -7 1
JZ 10        ; if not negative, jump to PRINT (word 10)
PUSH -1
MULT         ; negate: -7 * -1 = 7 - stack: 7
PRINT        ; word 10: print result
HALT
```

Output: 7

Note that the PRINT at word 10 is reached by two paths: the fall-through path (after the negation) and the direct jump (when the value was already non-negative). The two-word MULT instruction occupies words 8–9, so PRINT lands at word 10.

Maximum of two values


```

PUSH 14
PUSH 9
DUP          ; stack: 14 9 9
ST 0         ; save second value - stack: 14 9
GT           ; is 14 > 9? pushes 1 - stack: 1
JNZ 12       ; if yes, jump to PRINT_A (word 12)
LD 0         ; otherwise load the second value
PRINT       ; print 9
HALT

```

```

; --- word 12 ---
PUSH 14      ; reload first value
PRINT       ; print 14
HALT

```

Output: 14

4.3 Loops

All loops in the VM are built from a backward `JMP` combined with a conditional branch that exits when the loop condition is no longer satisfied.

Countdown from 5 to 1

```

PUSH 5          ; word 0: initial counter

; --- loop body (word 2) ---
DUP             ; word 2: duplicate for PRINT
PRINT          ; word 3: print current value
PUSH 1          ; word 4: decrement
SUB            ; word 6: counter - 1
DUP            ; word 7: duplicate for exit test
JZ 12           ; word 8: exit if counter reached 0
JMP 2           ; word 10: loop back

; --- word 12 ---
POP            ; discard the final 0
HALT

```

Output: 5, 4, 3, 2, 1

After the last decrement the counter reaches zero. `DUP` leaves a copy for `JZ`, which jumps to the `POP` at word 12, discarding the zero before `HALT`.

Summing 1 to N

This program accumulates the sum $1 + 2 + \dots + N$ into register `R1`.

```

PUSH 10
ST 0           ; R0 = N = 10
PUSH 0

```

```

ST 1          ; R1 = accumulator = 0
PUSH 1
ST 2          ; R2 = counter = 1

; --- loop body (word 12) ---
LD 2          ; push counter
LD 1          ; push accumulator
ADD           ; accumulator + counter
ST 1          ; store updated accumulator
LD 2          ; push counter
PUSH 1
ADD           ; counter + 1
ST 2          ; store updated counter
LD 2          ; push counter
LD 0          ; push N
GT            ; counter > N?
JNZ 12        ; if not, repeat
LD 1          ; push final sum
PRINT         ; prints 55
HALT

```

Output: 55

4.4 Functions

The `CALL` and `RET` instructions support structured function calls. By convention, arguments are pushed by the caller before `CALL` and the return value is left on the operand stack by the callee before `RET`.

Squaring a number

The function at word 6 pops one argument, squares it, and returns the result.

```

; --- caller ---
PUSH 9        ; word 0: argument
CALL 6        ; word 2: call square(); return addr = word
              4
PRINT         ; word 4: print return value (81)
HALT          ; word 5

; --- square(a): a^2 - starts at word 6 ---
DUP           ; word 6: a a
MULT          ; word 7: a*a
RET           ; word 8

```

Output: 81

Recursive factorial

The following program computes $5!$ using a recursive function. The factorial function calls itself for $n > 1$ and returns 1 as the base case for $n \leq 1$.

```

; --- caller ---
PUSH 5          ; word 0
CALL 6          ; word 2: call fact(); return addr = word 4
PRINT          ; word 4
HALT           ; word 5

; --- fact(n) - starts at word 6 ---
DUP            ; word 6:  n n
PUSH 1
GT            ; word 8:  n (n>1)?
JZ 22         ; word 10: if n<=1 jump to base case (word
                22)

; recursive case
DUP            ; word 12: n n
PUSH 1
SUB           ; word 14: n (n-1)
CALL 6        ; word 16: fact(n-1); return addr = word 18
MULT          ; word 18: n * fact(n-1)
RET           ; word 19

; base case (word 22): n is on the stack, value is 1
POP           ; word 22: discard n
PUSH 1        ; push 1
RET           ; word 24

```

Output: 120

Each recursive call saves a return address on the call stack and leaves the partial result on the operand stack. When the base case fires, the chain of `MULT` operations unwinds, accumulating $5 \times 4 \times 3 \times 2 \times 1 = 120$.

4.5 Register-Based Patterns

Registers shine in programs where values must survive multiple stack operations without being continuously re-pushed. The examples below illustrate two common idioms.

Swapping two values via a register

```

PUSH 42
PUSH 99
ST 0          ; R0 = 99
              ; stack: 42
ST 1          ; R1 = 42
              ; stack: (empty)
LD 0          ; push 99
LD 1          ; push 42
PRINT         ; prints 42
PRINT         ; prints 99

```

HALT

Output: 42, 99

Loop with a persistent accumulator

The Fibonacci sequence up to the N -th term demonstrates the register idiom for carrying state across loop iterations. Two registers hold the two previous terms; a third holds the loop counter.

```
PUSH 0
ST 0          ; R0 = F(0) = 0
PUSH 1
ST 1          ; R1 = F(1) = 1
PUSH 8
ST 2          ; R2 = remaining iterations

LD 0
PRINT         ; print F(0) = 0
LD 1
PRINT         ; print F(1) = 1

; --- loop (word 18) ---
LD 0
LD 1
ADD           ; F(n) = F(n-1) + F(n-2)
ST 3         ; R3 = new term
LD 1
ST 0         ; R0 = old F(n-1)
LD 3
ST 1         ; R1 = new F(n)
LD 3
PRINT        ; print new term
LD 2
PUSH 1
SUB
ST 2         ; decrement counter
LD 2
JNZ 18       ; repeat if counter > 0
HALT
```

Output: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

4.6 Patterns and Idioms Summary

The examples above recur throughout non-trivial VM programs. The table below collects the most useful idioms for quick reference.

Goal	Idiom
Test top of stack without consuming it	DUP before comparison
Loop back unconditionally	JMP <loop-start>
Exit a loop when counter reaches zero	DUP / JZ <exit>
Pass an argument to a function	PUSH <val> / CALL <addr>
Return a value from a function	Leave it on the stack before RET
Save a value across stack operations	ST <reg> then LD <reg>
Negate a boolean result	NOT
Implement $a \leq b$	GT NOT
Implement $a \neq b$	EQ NOT

5 Implementation Notes

This section examines a selection of non-obvious decisions made during the implementation: why certain things were designed the way they were, where the current code contains latent defects, and what the consequences of those choices are at runtime. The intent is not to enumerate every line of code, but to highlight the points where the implementation departs from a naive first approach and the points where it does not yet go far enough.

5.1 The push and pop Primitives

A key architectural decision is that `push` and `pop` live in `vm.c` rather than in any operation module. Every handler in `maths.c`, `logic.c`, and so on calls these two functions rather than manipulating `vm->sp` directly. This centralises the overflow and underflow checks in exactly one place each, which means a bug in the bounds check — or a future change to the stack size — needs to be fixed in a single location.

The underflow guard in `pop` deserves particular attention:

```
int pop(VM *vm) {
    if (vm->sp == 0) {
        printf("Stack underflow\n");
        vm->running = 0;
        return 0;
    }
    return vm->stack[--vm->sp];
}
```

When underflow is detected, the function sets `running = 0` and returns 0. Returning a sentinel is necessary because `pop` has a non-void return type: the caller will assign the result to a local variable regardless. Returning 0 instead of leaving the variable uninitialised prevents undefined behaviour in the caller, even though the VM will halt before that value is used again. This is a deliberate defensive choice.

5.2 A Latent Bug in `op_div`

The division handler in `maths.c` contains a structural defect that is worth examining in detail, because it illustrates a class of error that arises whenever error-handling and computation are interleaved without `else` guards:

```
void op_div(VM *vm) {
    if (vm->sp < 2) {
        printf("Stack underflow\n");
        vm->running = 0;
    }

    int b = pop(vm);          // (1)

    if (b == 0) {
        printf("Division by zero error\n");
        vm->running = 0;
    }

    int a = pop(vm);         // (2)
    push(vm, a / b);         // (3)
}
```

There are two problems here. First, the underflow check at the top sets `running = 0` but does not `return`. Execution falls through to line (1), which calls `pop` on a stack that may have zero or one element. The `pop` call will detect underflow a second time and return 0, but this double-fault produces two error messages and leaves the function executing further than it should.

Second, the division-by-zero check at the middle similarly has no `return`. When `b == 0`, the VM is halted but execution continues: `a` is popped at line (2) and `a / b` is evaluated at line (3). In C, integer division by zero is *undefined behaviour* — the compiler is permitted to generate any code it likes, including a program crash or a silent wrong answer. The fact that `running` has already been set to 0 does not protect against this: the undefined behaviour occurs at the point of the division, before the loop in `vm_run` has a chance to check the flag.

The correct implementation uses `else` branches and early `returns` to ensure these paths are mutually exclusive:

```
void op_div(VM *vm) {
    if (vm->sp < 2) {
        printf("Stack underflow\n");
        vm->running = 0;
        return;
    }
    int b = pop(vm);
    if (b == 0) {
        printf("Division by zero\n");
        vm->running = 0;
        return;
    }
    int a = pop(vm);
    push(vm, a / b);
}
```

```

    }
    int a = pop(vm);
    push(vm, a / b);
}

```

Every other arithmetic handler (`op_add`, `op_sub`, `op_mult`, `op_mod`) uses a proper `if/else` structure and is not affected by this issue.

5.3 A Latent Bug in `op_store`

The register store handler has an analogous problem:

```

void op_store(VM *vm) {
    int reg = vm->program[vm->ip++];

    if (reg < 0 || reg >= 16) {
        printf("Invalid register: %d\n", reg);
        vm->running = 0;
    }

    if (vm->sp == 0) {
        printf("Stack underflow\n");
        vm->running = 0;
    }

    vm->registers[reg] = pop(vm);    // always reached
}

```

Neither error check is followed by a `return`. If the register index is invalid, execution continues to the `pop` and then to the array write `vm->registers[reg]`, which is an out-of-bounds write — undefined behaviour with potential memory corruption. If the stack is empty, `pop` returns its sentinel 0 and the (possibly invalid) register is written anyway.

The counterpart `op_load` handles this correctly with an `else` clause, so the inconsistency between the two register operations is likely an oversight rather than a design choice.

5.4 Flat Integer Bytecode

Programs are stored as plain `int` arrays. Opcodes and operands share the same representation; the only distinction between an opcode word and a data word is positional context — the VM knows from the mnemonic table how many words following an opcode are operands rather than the next instruction.

This encoding is extremely simple to generate (the assembler is around 60 lines of C) and simple to execute (the fetch stage is a single array index). The cost is that the bytecode is not self-describing: there is no way to scan the array and determine instruction boundaries without executing from the start. A disassembler, for instance, would need to replicate the same operand-count table used by the assembler.

A more conventional approach would encode the operand count into the opcode word itself — for example, using the high 16 bits for the opcode and the low 16 bits for an immediate value. This would allow random-access disassembly but would complicate the fetch loop. For a VM of this scale, the flat array is the right trade-off.

5.5 Zero-Initialisation of the VM Struct

Every REPL session creates a fresh VM with:

```
VM vm = {0};
```

The `= {0}` initialiser is a C11 idiom that zero-initialises every field of the struct: `sp`, `ip`, `csp`, all 256 stack slots, all 16 registers, and all 128 call stack slots. This matters for correctness in two ways. First, `sp = 0` and `ip = 0` mean the stack and instruction pointer are in valid starting states without any explicit assignment. Second, all 16 registers are initialised to zero, which matches the documented behaviour that “registers hold their values across instructions and are initialised to zero at VM startup.” An uninitialised struct would contain whatever happened to be in that stack frame’s memory, producing non-deterministic register reads.

5.6 Separation of the Operand and Call Stacks

A common simplification in toy VM implementations is to use a single stack for both operands and return addresses, with `CALL` pushing the return address onto the operand stack and `RET` popping it. This VM uses two completely separate arrays instead: `stack[256]` for operands and `call_stack[128]` for return addresses, each with its own pointer (`sp` and `csp`).

The two-stack design has a concrete correctness advantage: a function’s return address cannot be accidentally corrupted by arithmetic operations on the operand stack, and a stack underflow in user code cannot silently consume a return address. The operand stack and the call stack fail independently with distinct error messages. This also mirrors the design of real processors, which maintain a hardware stack pointer separate from the general-purpose registers.

The cost is a modest increase in the size of the VM struct (128 additional ints, or 512 bytes on a 32-bit platform), which is negligible in practice.

5.7 No Bounds Checking on Jump Targets

The jump instructions (`JMP`, `JZ`, `JNZ`, `CALL`) write an arbitrary integer directly into `ip`:

```
void op_jump(VM *vm) {
    int addr = vm->program[vm->ip++];
    vm->ip = addr;
}
```


No check is performed to verify that `addr` lies within the bounds of the program array. A jump to a negative index or an index beyond the end of the allocated bytecode reads from memory outside the `Program` buffer — undefined behaviour. In practice, the `Program` struct allocates a fixed `words[1024]` array on the stack in `main`, so an out-of-bounds jump reads from adjacent stack memory rather than segfaulting, which makes the resulting behaviour difficult to diagnose.

A minimal fix would be to pass the program length into `vm_run` and check the target against it before assigning to `ip`. The current design omits this check in the interest of simplicity, which is an acceptable trade-off for a development tool where the programmer controls the bytecode.

5.8 The REPL’s Auto-Reset Behaviour

After each successful run, the REPL resets the program buffer automatically:

```
prog.length = 0;
```

This is a deliberate usability decision. Without it, a blank-line trigger would re-execute the same program on every subsequent blank line, which is surprising behaviour. The auto-reset means each execution is a clean slate, matching the expectation that the REPL is a scratch pad for experimentation rather than a persistent program store. The explicit `RESET` command exists for cases where the user wants to clear the buffer *without* running first.

6 Comparison with Real-World Virtual Machines

The VM described in this paper does not exist in a vacuum. Stack-based virtual machines are one of the most studied and widely deployed software artefacts in computing — the Java Virtual Machine [4] has executed production code since 1995, CPython’s bytecode interpreter [10] underpins virtually all Python in the world, and WebAssembly [11, 12] has introduced a new formally specified VM into every modern web browser. Situating this project against those systems clarifies which design choices are universal and which are specific to the constraints and goals of this implementation.

6.1 The Java Virtual Machine

The JVM is perhaps the closest spiritual relative of the VM built here. It is explicitly stack-based: the JVM specification defines an operand stack per stack frame, and most bytecode instructions implicitly read from and write to the top of that stack. The `iadd` instruction, for example, pops two integers and pushes their sum — identical in semantics to this project’s `ADD`.

The differences are instructive. The JVM operates on *typed* values: its operand stack is not a flat array of integers but a sequence of typed slots, and the bytecode verifier [13] statically checks that no instruction is ever applied to a

value of the wrong type before the program begins executing. This project’s VM is entirely untyped — every value is a plain C `int`, and there is no mechanism to distinguish a boolean, a character, or a pointer from an arithmetic result. The simplification is appropriate for a teaching tool but would be the first thing to address in a production system.

The JVM also compiles to native machine code at runtime via a Just-In-Time (JIT) compiler [14], which profiles hot code paths and replaces the interpreter loop with optimised native instructions. This project interprets every instruction individually, which is simpler to implement and reason about but substantially slower for compute-intensive programs. The gap between a naive interpreter and a JIT-compiled loop is typically one to two orders of magnitude on arithmetic-heavy workloads.

6.2 CPython

CPython, the reference implementation of Python, uses a stack-based bytecode interpreter that is structurally very similar to the one built here. Python source is compiled to `.pyc` bytecode files, which are then interpreted by a central `switch` loop in `ceval.c`. That loop, like `vm_run` in this project, dispatches on an integer opcode and calls the appropriate handler.

The most striking difference is complexity of state. CPython’s evaluation frame carries not just an operand stack and an instruction pointer, but also a reference to the local variable array, the enclosing global and builtin namespaces, the current exception state, and enough metadata to produce meaningful tracebacks on error. This project’s VM struct is around ten fields; CPython’s `PyInterpreterFrame` has closer to twenty, each backed by a rich object system.

CPython also uses reference counting [15] with a cycle-detecting garbage collector [16] to manage heap-allocated objects. This project allocates nothing on the heap at runtime: all VM state lives in the VM struct on the C stack, and values are plain integers. The absence of heap allocation and garbage collection is both the biggest simplification and the biggest limitation — the VM cannot represent strings, arrays, or any data structure larger than a single integer.

6.3 WebAssembly

WebAssembly (Wasm) is a binary instruction format designed as a portable compilation target for languages like C, C++, and Rust. Its execution model is stack-based in the same sense as this project and the JVM: instructions implicitly consume and produce values on a virtual stack. A Wasm `i32.add` instruction is semantically equivalent to this project’s `ADD`.

What sets Wasm apart is its formal specification. Every aspect of the execution model — the type system, the stack discipline, the memory model, the trap conditions — is defined in a mathematical notation that admits formal proof. The Wasm type system is strong enough that a compliant Wasm engine can verify a module’s safety in a single linear pass before executing a single instruction. In contrast, this project has no type system and no static verification

phase: a program that pushes a register index as data and then jumps to it as an address will execute without complaint.

Wasm also defines a *linear memory* model – a flat, byte-addressable array that the guest program can read and write with load and store instructions. This project’s memory model is the operand stack and sixteen integer registers; there is no byte-addressable heap. Adding a memory model analogous to Wasm’s linear memory would be the most significant architectural extension possible within the existing design.

6.4 Summary of Differences

Property	This VM	JVM	CPython	Wasm
Architecture	Stack-based	Stack-based	Stack-based	Stack-based
Value types	int only	Typed slots	Python objects	i32/i64/f32/f64
Static verification	None	Bytecode verifier	None	Full type check
Execution model	Interpreted	JIT compiled	Interpreted	Interpreted / JIT
Memory model	Stack + 16 regs	Heap (GC)	Heap (RC + GC)	Linear memory
Heap allocation	None	Yes (GC)	Yes (RC)	Yes (manual)
Call frames	Single shared stack	Per-frame stack	Per-frame + locals	Per-frame stack
External dependencies	None	JDK runtime	CPython runtime	Host environment
Lines of C source	~600	~350,000	~500,000	varies

The table makes clear that this project occupies the simplest possible position on almost every axis. That is not a shortcoming — it is the consequence of optimising for a different goal. The JVM, CPython, and Wasm are each production systems that must handle untrusted code safely, integrate with large ecosystems, and deliver competitive performance. This VM must be readable, buildable in a single `gcc` invocation, and small enough that every design decision can be explained in a short paper. On those criteria it succeeds completely.

The shared stack-based execution model across all four systems also demonstrates something more fundamental: the stack machine is not merely a pedagogical device [17, 18]. It is a genuinely useful abstraction that real-world language designers and standards bodies have independently converged on, because its simplicity of specification and implementation outweighs the cost of the occasional extra `DUP` instruction [19].

7 Conclusions

This project has demonstrated that a functional stack-based virtual machine complete with a custom instruction set, an assembler, and an interactive REPL can be implemented in roughly 600 lines of C with no dependencies beyond the standard library. The result is a system small enough to be read and understood in its entirety, yet expressive enough to evaluate arithmetic expressions, implement

conditional logic, execute recursive function calls, and maintain persistent state across loop iterations using sixteen general-purpose registers.

Several design decisions proved their value through the implementation process. Centralising `push` and `pop` in `vm.c` ensured that stack-safety checks exist in exactly one place each, eliminating an entire class of potential inconsistency. Separating the operand and call stacks prevented return addresses from being silently corrupted by arithmetic operations — a subtle correctness property that real processors enforce in hardware. Zero-initialising the VM struct with `= {0}` eliminated a category of non-deterministic behaviour at negligible cost. And confining each opcode category to its own translation unit kept the codebase navigable throughout development.

The implementation notes in Section 5 are equally instructive for what they reveal about the cost of incomplete error handling. The latent bugs in `op_div` and `op_store`, both caused by missing `return` statements after setting `running = 0` — illustrate a recurring pitfall in C: the language provides no mechanism to enforce that an error branch terminates. In a larger system these defects could manifest as memory corruption or undefined behaviour that is difficult to reproduce. The correct pattern, demonstrated in every other handler, is to use early returns and `else` guards so that error paths and computation paths are structurally mutually exclusive.

The comparison with the JVM, CPython, and WebAssembly in Section 6 places this project in a wider context. All four systems share the same stack-based execution model, which confirms that the stack machine is not merely a pedagogical simplification — it is an architectural choice that production language designers have independently adopted because its specification and implementation costs are genuinely low. What distinguishes the production systems is the machinery layered on top: type systems, bytecode verifiers, garbage collectors, just-in-time compilers, and formally specified memory models. This VM occupies the simplest possible position on every one of those axes, and that simplicity is the point. Every layer that was deliberately omitted is a layer whose absence makes the remaining design easier to reason about.

Future work could extend the VM in several directions. Adding a linear byte-addressable memory region analogous to WebAssembly’s linear memory would allow the VM to represent strings and arrays, removing the most significant expressive limitation of the current design. Introducing a type tag in the high bits of each stack word would enable basic runtime type checking without changing the integer representation of values. Bounds-checking jump targets before assigning to `ip` would close the only remaining source of undefined behaviour. Each of these extensions can be made incrementally, touching at most the files identified in the separation-of-concerns analysis, which validates the modularity of the original architecture.

The virtual machine would also benefit from save/load using `.txt` files in order to save previously-written programs; as well as a separate interface using C++ library like SFML to provide cleaner visual identity and editing capabilities to the software.

Ultimately, this project achieves the goal stated in its motivation: to make

explicit every design decision that a language runtime ordinarily hides. The fetch–decode–execute loop, the implicit operand convention of a stack machine, the mechanics of a call stack, and the encoding of a flat integer bytecode stream are no longer abstractions. They are concrete, readable, and — intentional bugs aside — correct.

References

- [1] ISO. *ISO/IEC 9899:2011 Information technology — Programming languages — C*. International Standard ISO/IEC 9899:2011. International Organization for Standardization, 2011.
- [2] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. 6th. Cambridge, MA: Morgan Kaufmann, 2017.
- [3] Jan Lukasiewicz. *Elements of Mathematical Logic*. Oxford: Pergamon Press, 1963.
- [4] Tim Lindholm et al. *The Java Virtual Machine Specification, Java SE 8 Edition*. Java SE 8. Addison-Wesley Professional, 2014.
- [5] Microsoft Corporation. *Common Language Runtime (CLR) Overview*. .NET Documentation. 2023. URL: <https://learn.microsoft.com/en-us/dotnet/standard/clr>.
- [6] David A. Patterson and David R. Ditzel. “The Case for the Reduced Instruction Set Computer”. In: *ACM SIGARCH Computer Architecture News* 8.6 (1980), pp. 25–33.
- [7] Kitware Inc. *CMake Documentation*. Version 3.28. 2023. URL: <https://cmake.org/documentation/>.
- [8] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd. Boston, MA: Addison-Wesley, 2006.
- [9] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. 2nd. Burlington, MA: Morgan Kaufmann, 2011.
- [10] Guido van Rossum, Barry Warsaw, and Nick Coghlan. *CPython internals: A ten-hour codewalk through the Python interpreter source code*. PyCon 2014. 2014. URL: https://www.youtube.com/watch?v=LhadeL7_EIU.
- [11] Andreas Haas et al. *Bringing the Web up to Speed with WebAssembly*. Tech. rep. ACM, 2017, pp. 185–200.
- [12] WebAssembly Community Group. *WebAssembly Specification*. W3C WebAssembly Working Group. 2023. URL: <https://webassembly.github.io/spec/core/>.
- [13] Stephen N. Freund and John C. Mitchell. “A Type System for Object Initialization in the Java Bytecode Language”. In: *Proceedings of the 13th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, 1998, pp. 310–327.

- [14] L. Peter Deutsch and Allan M. Schiffman. “Efficient Implementation of the Smalltalk-80 System”. In: *Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (1984), pp. 297–302.
- [15] George E. Collins. “A Method for Overlapping and Erasure of Lists”. In: *Communications of the ACM* 3.12 (1960), pp. 655–657.
- [16] Richard Jones, Antony Hosking, and Eliot Moss. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. 1st. Chapman and Hall/CRC, 2011.
- [17] James E. Smith and Ravi Nair. *Virtual Machines: Versatile Platforms for Systems and Processes*. San Francisco, CA: Morgan Kaufmann, 2005.
- [18] Philip Koopman. “Stack Computers: The New Wave”. In: *Ellis Horwood Books in Computing Science* (1989).
- [19] Yunhe Shi et al. “Virtual Machine Showdown: Stack Versus Registers”. In: *ACM Transactions on Architecture and Code Optimization* 4.4 (2008), pp. 1–36.